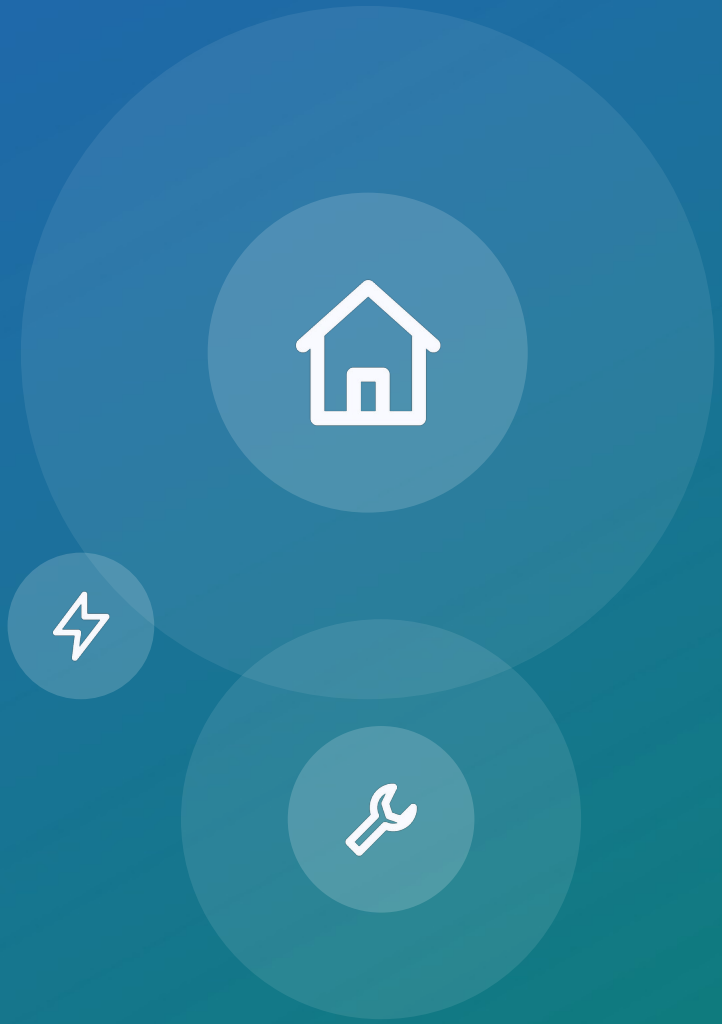
DESIGN REVIEW · UPDATED 18 OCTOBER 2026

Ji-Hajoor

Architecture, data, deployment and security in pictures



On-demand home-services marketplace · branch review/2026-10-18-open-items

UPDATE · 30 OCTOBER 2026

What changed on 30 October 2026

Security review

- Web sessions: refresh token in an HttpOnly, Secure, SameSite=Strict cookie
- Passwords: 10+ characters, letter and digit; common, sequence and personal ones refused
- Rate limits keyed on Railway’s X-Real-IP; https-only image links; masked query strings in logs
- npm audit: 0 vulnerabilities in all three apps

Reviews engine

- Review prompt after payment, one reminder after 24 h (ReviewReminder: 14 event types now)
- Public page per verified worker: /public/workers/{id} and its reviews
- Workers reply to reviews; share-my-profile card in the partner app
- Booking-flow CI now covers prompt, moderation, public page and reply

Names used everywhere

- Apps: Ji-Hajoor (customers), Ji-Hajoor Pro (workers), Ji-Hajoor Admin (staff, My agency)
- People: customer, worker (Provider in code), agency (Tenant in code)
- Agencies invite workers; agency share 0–50% of the worker’s net

Read next

- JiHajoor_Tech_Stack.pptx: current stack and security state
- JiHajoor_Pitch_Deck, Revenue_Model, Brand_and_Marketing
- User guides: docs/guides/; PDFs in docs/pdf/
- Security findings later in this deck show their state on 18 October

AT A GLANCE

An event-driven platform: Railway today, AWS-ready

20

services in 5 Java 25 / Boot 4.1 JVMs

3+2

React web apps plus iOS and Android shells

13

Kafka topics fed by a transactional outbox

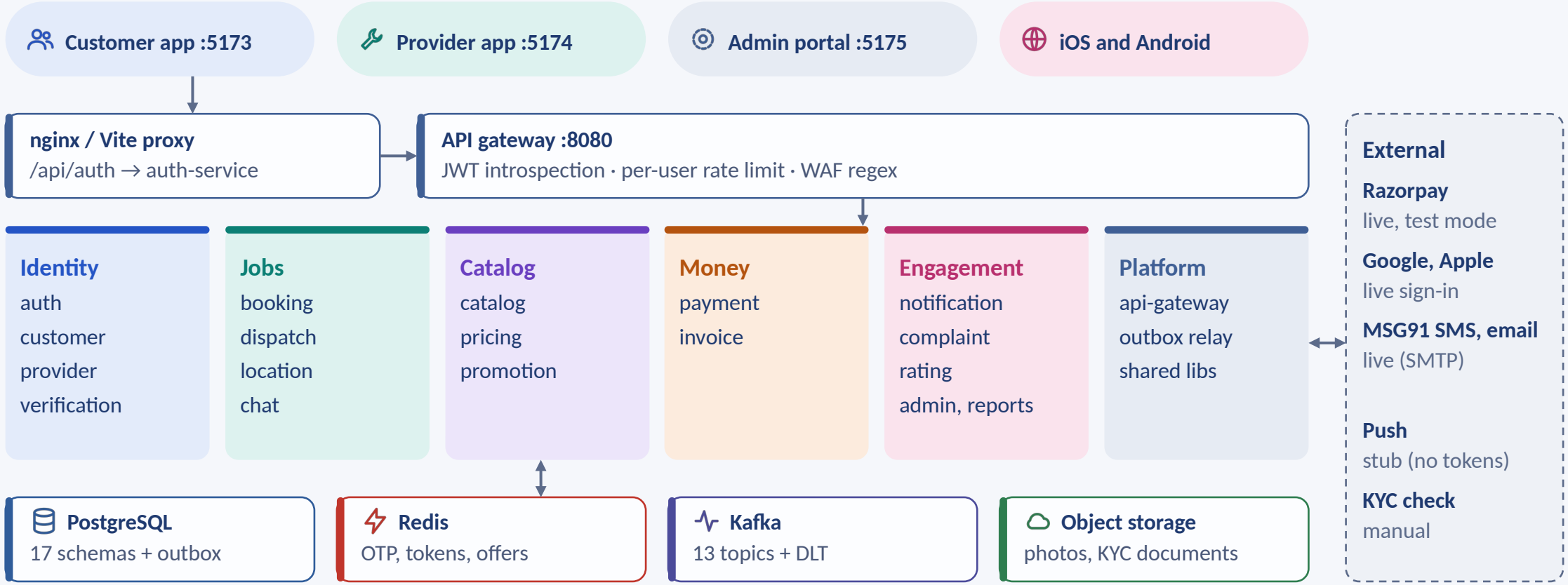
17

Postgres schemas, one per service

Live: Razorpay (test mode), Google/Apple, MSG91 OTP, SMTP email, S3. **Stubs:** push, notice SMS, background checks, geocoding.

ARCHITECTURE · UNIFIED VIEW

One picture: apps, edge, six domains, data, partners



ARCHITECTURE · SERVICE MAP

Twenty services in six colour-coded domains

Identity and people

auth 8081 · customer 8082
provider 8083 · verification 8094

Sign-in, profiles, agencies, KYC and background checks

Jobs

booking 8084 · dispatch 8087
location 8092 · chat 8093

Booking state machine, matching and offers, live position, chat

Catalog and price

catalog 8085 · pricing 8086
promotion 8098

Categories, estimates with cached config, coupons

Money

payment 8088 · invoice 8089

Razorpay charges, refunds, settlements, PDF invoices

Engagement and ops

notification 8090 · complaint 8091
rating 8097 · admin 8095 · reporting 8096

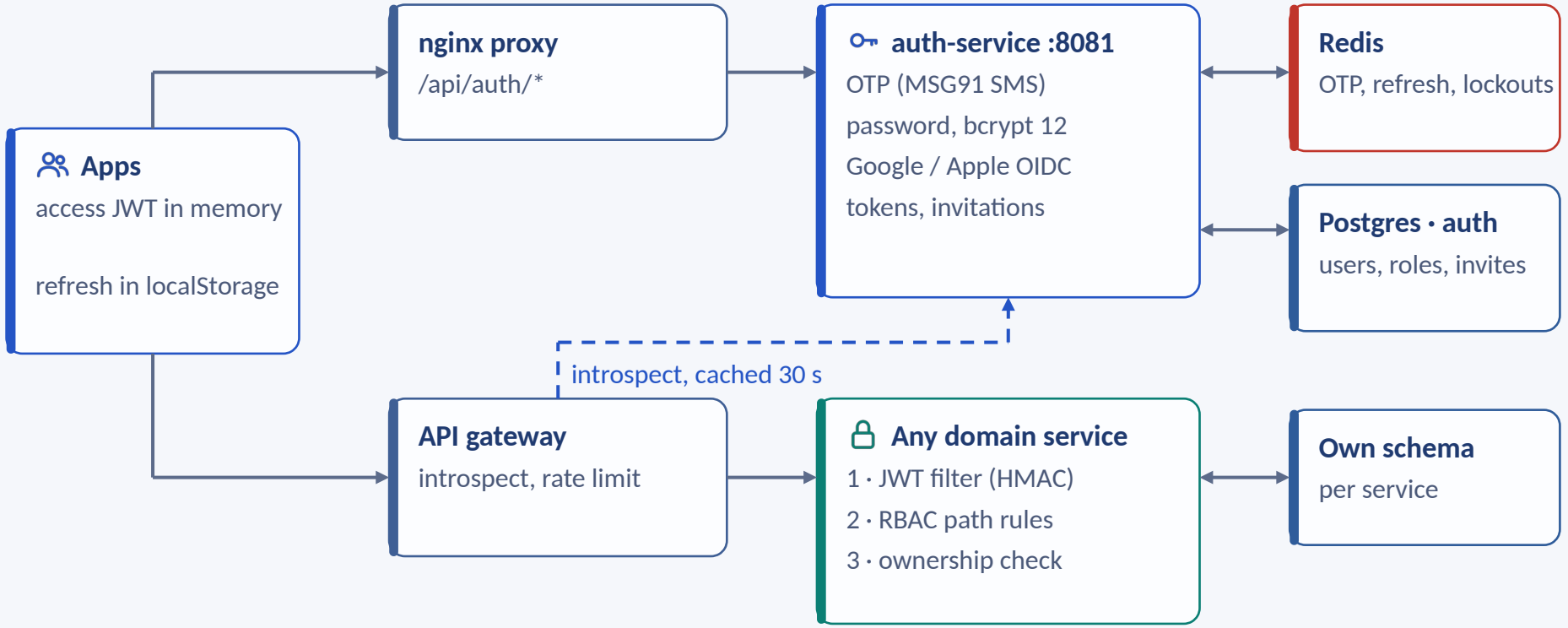
Notices, complaints with SLA sweep, reviews, audit log

Platform

api-gateway 8080
outbox-processor 8099

Routing, auth, rate limits, outbox relay, shared libraries

Every API call is checked three times



Tokens

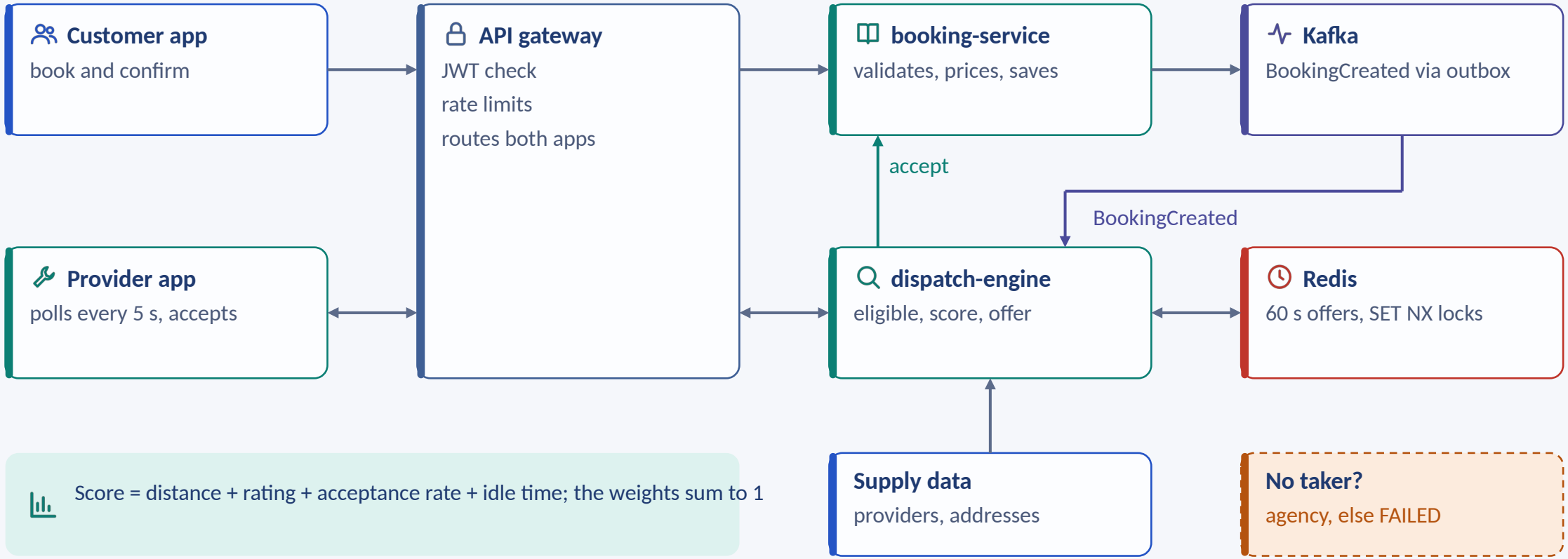
Access JWT, 15 min, HMAC

Refresh: opaque, 30 days, rotated

Replay revokes the whole family

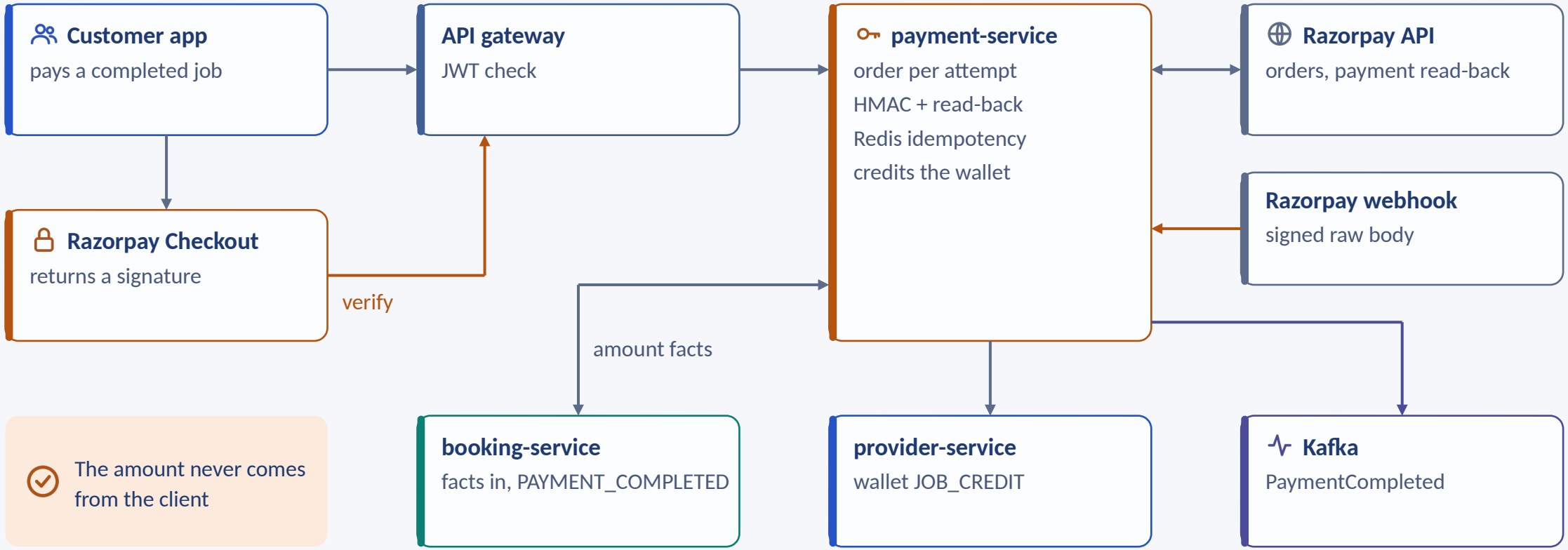
Staff roles by invitation only

Bookings flow out through Kafka; offers live in Redis



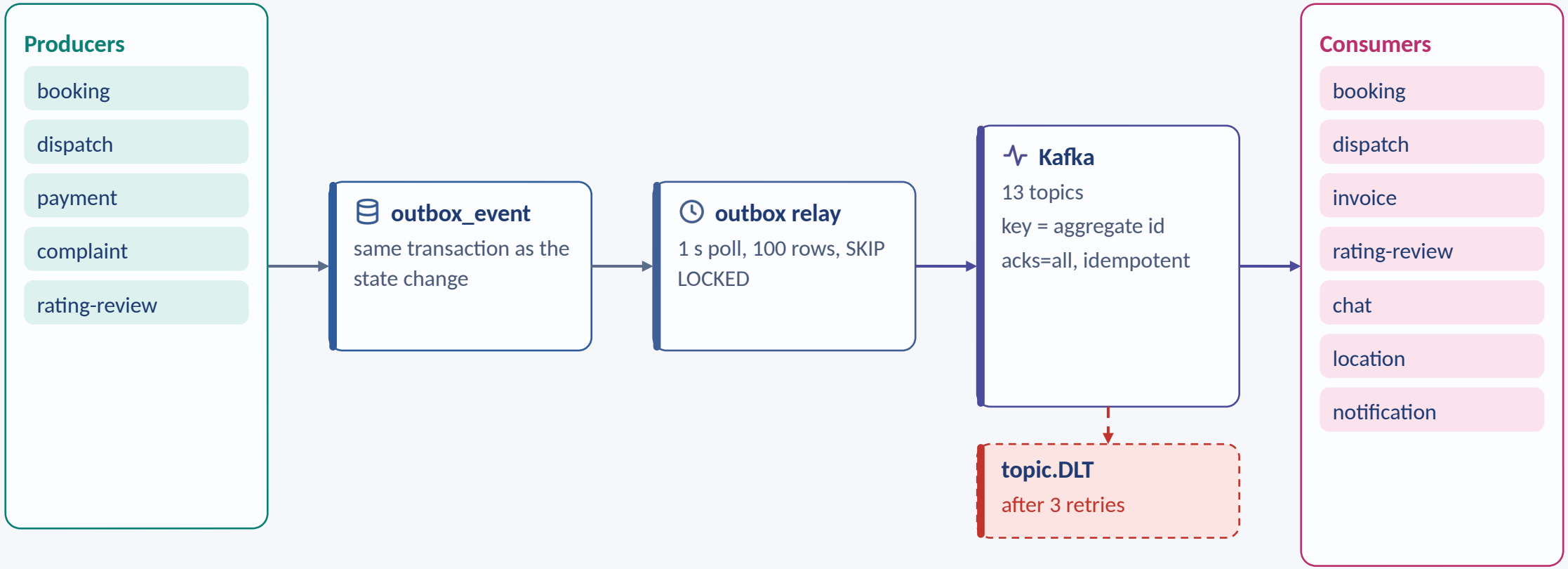
ARCHITECTURE · PAYMENTS

Razorpay confirms the charge twice before money counts



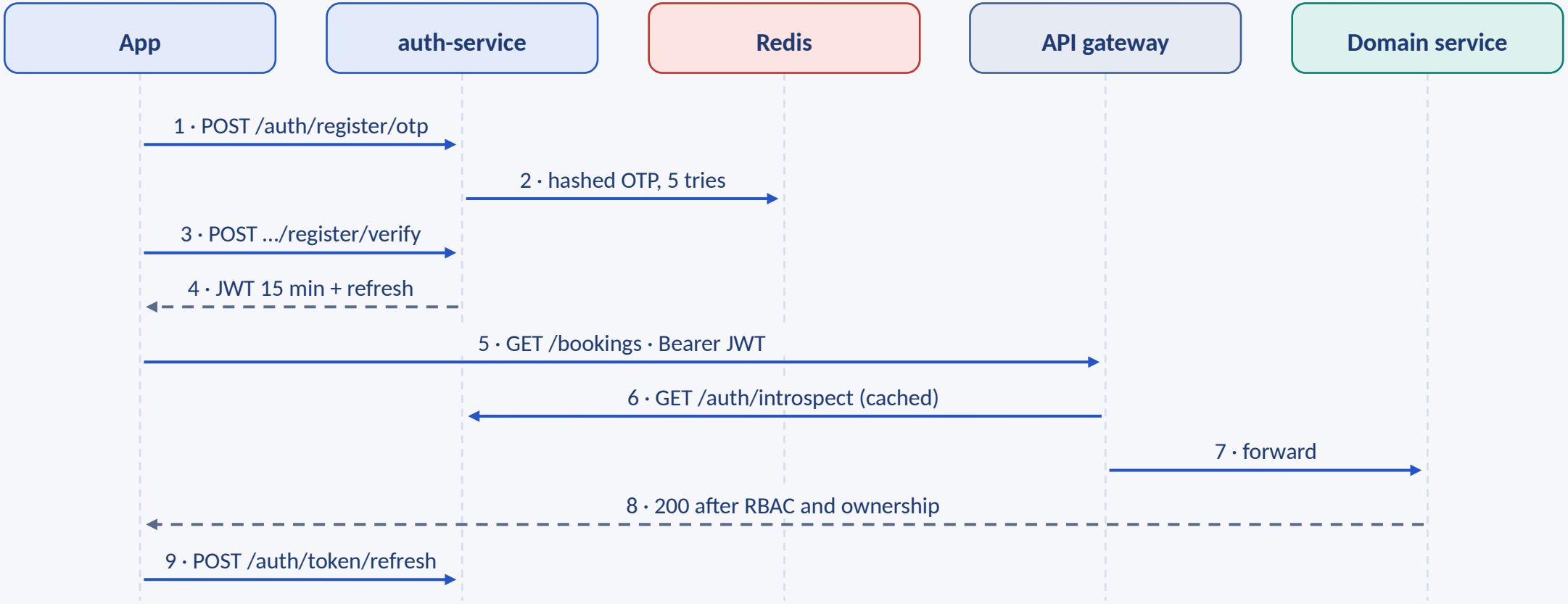
ARCHITECTURE · EVENTS

Events are as durable as the writes behind them



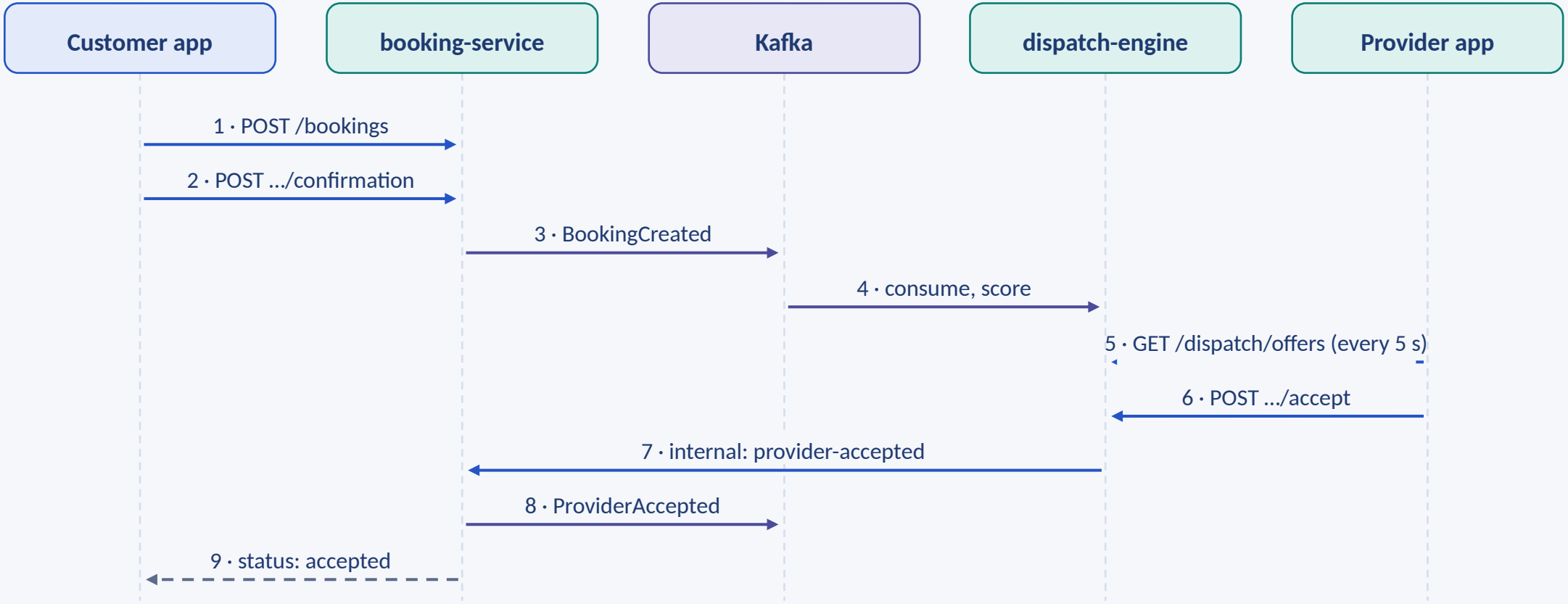
SEQUENCE · SIGN-IN AND API CALL

OTP sign-in, an API call, then a silent refresh



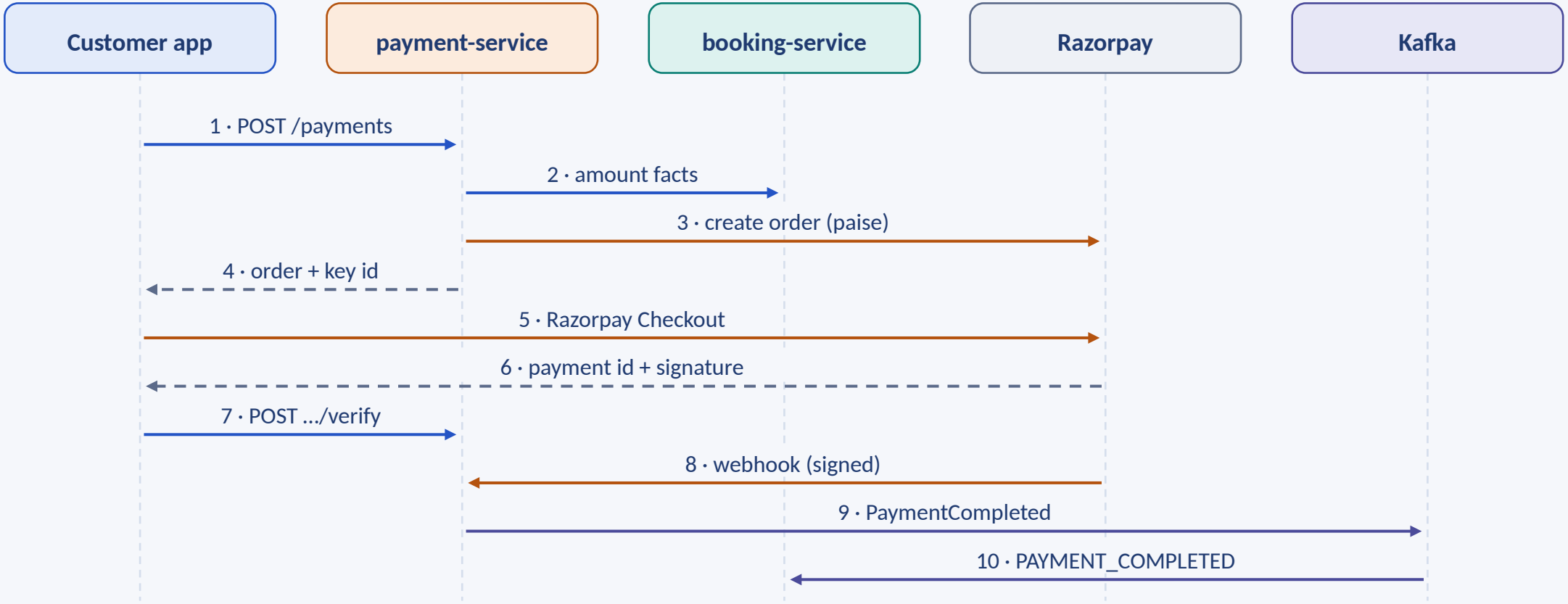
SEQUENCE · BOOKING TO ACCEPTANCE

Offers go to one provider until someone accepts



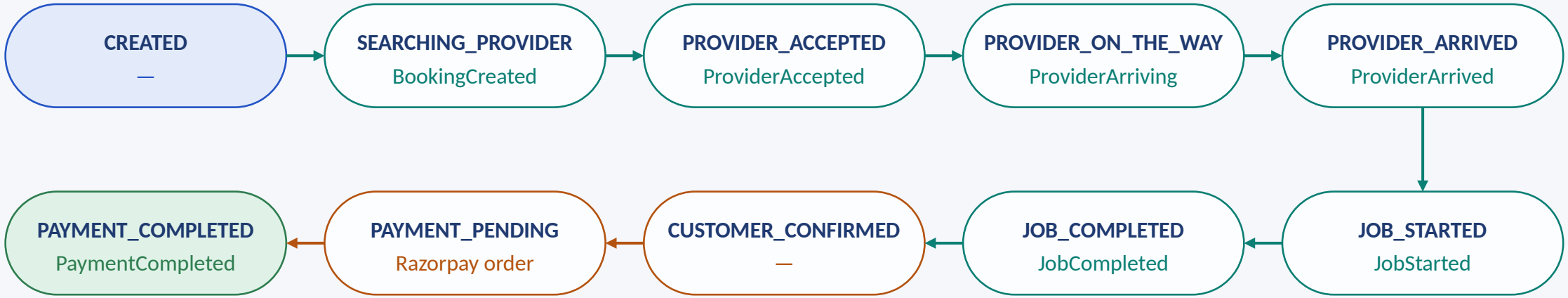
SEQUENCE · PAYMENT

Verify and webhook settle each payment once



BOOKING STATE MACHINE

Ten states on the happy path, five ways off it



Side paths

AWAITING_ASSIGNMENT
no taker, an agency covers it

SEARCHING_FAILED
no taker, no agency

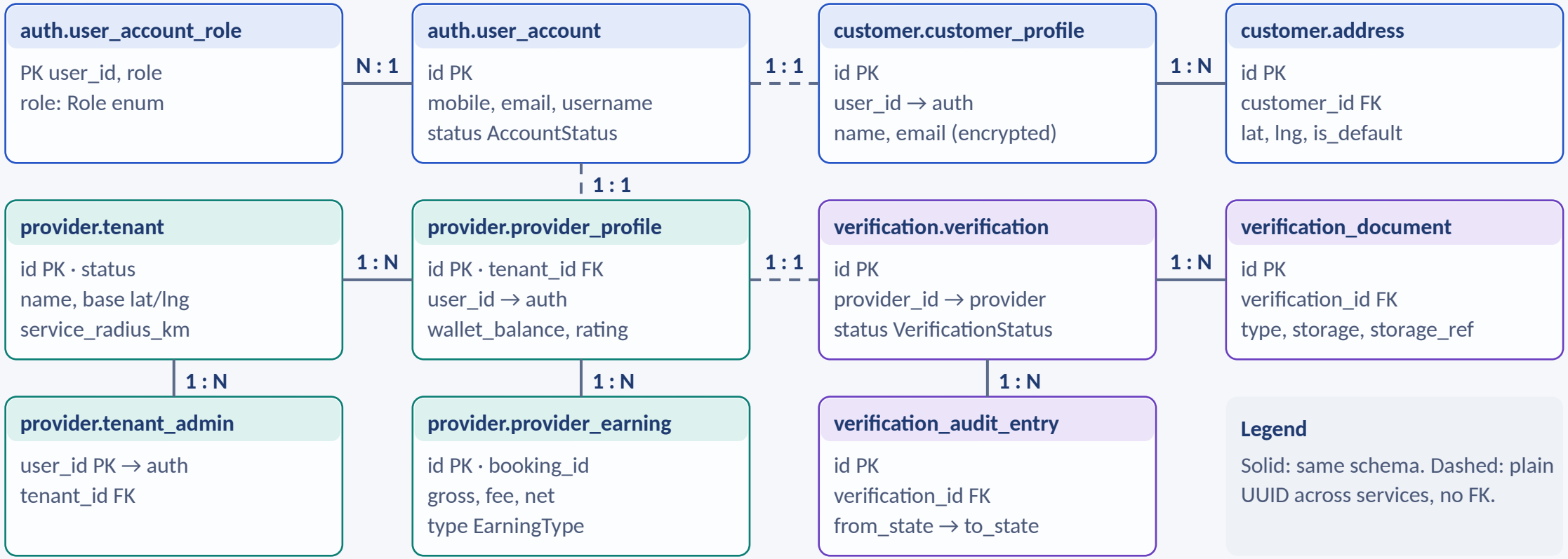
ADDITIONAL_QUOTE_REQUIRED
parts quote to approve

DISPUTED → REFUNDED
complaint upheld

CANCELLED
any time before the job

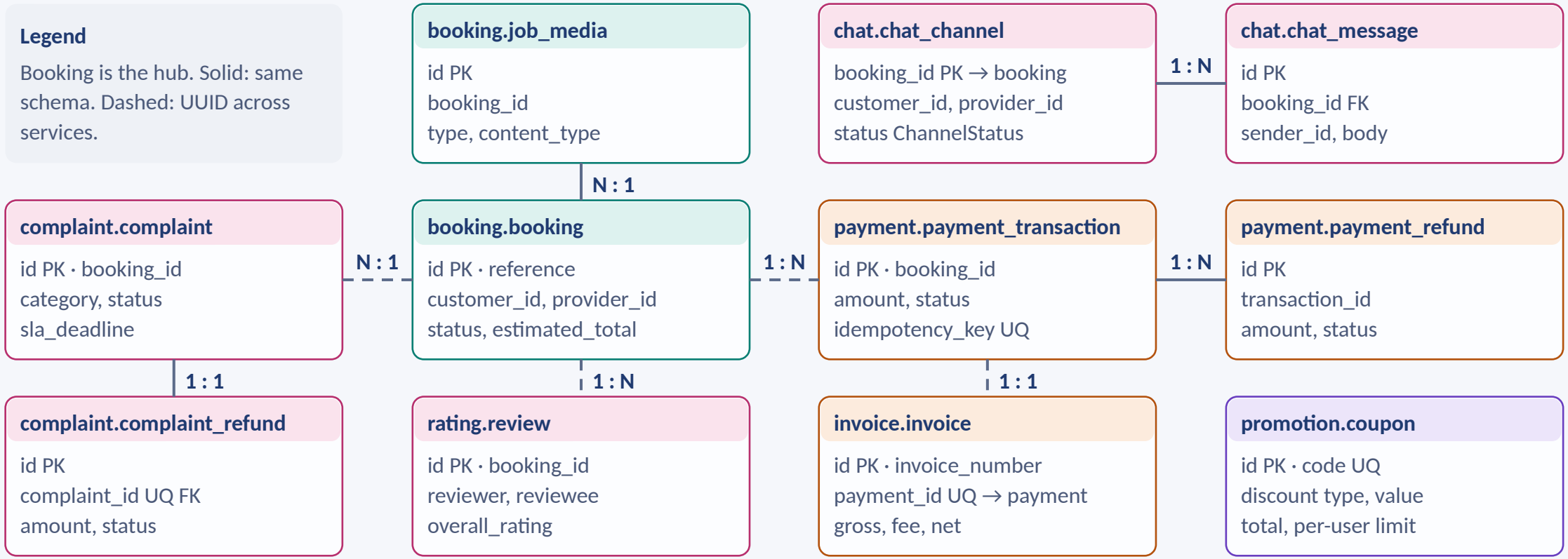
LOGICAL DATA MODEL · PEOPLE AND SUPPLY

Accounts, customers, providers and agencies



LOGICAL DATA MODEL · JOBS AND MONEY

Everything after the request hangs off the booking



WHERE STATE LIVES

Postgres keeps records; Redis keeps moments

PostgreSQL

- auth
- customer
- provider
- verification
- catalog
- pricing
- promotion
- booking
- dispatch
- location
- payment
- invoice
- notification
- complaint
- chat
- rating
- admin
- outbox

Redis

- auth:otp:session:{phone}
- auth:refresh:{sha256}
- dispatch:offer:booking:{id}
- dispatch:lock:provider:{id}
- location:latest:{providerId}
- payment:idem:{key}
- gw:ratelimit:user:{sub}

Kafka

- BookingCreated
- ProviderAccepted
- ProviderRejected
- JobStarted
- JobCompleted
- PaymentCompleted
- BookingCancelled
- ReviewSubmitted
- ComplaintCreated
- + 4 more

READINESS

The happy path works end to end; a few edges are stand-ins

Works end to end

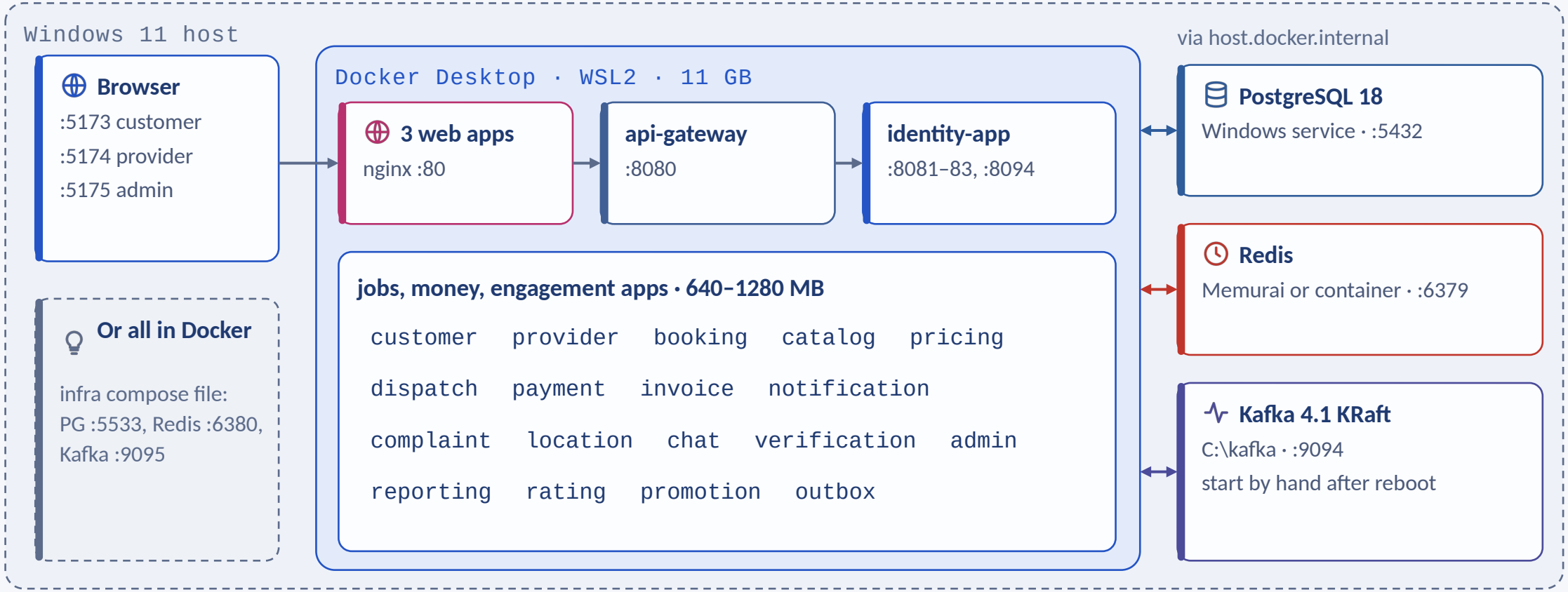
- OTP, email, Google and Apple sign-in
- Book, dispatch, accept, start, complete
- Razorpay Checkout, verify, webhook
- Outbox to Kafka to consumers
- Agency fallback and Tenant Portal
- Staff invitations, verification queue

Still stand-ins or open

- Push: no device tokens
- Notice SMS only logs
- Payouts are simulated
- Refunds don't debit the wallet
- Coupon redemption never called
- No provider online/offline state

LOCAL DEPLOYMENT · TOPOLOGY

Apps run in Docker; data services run on the host



LOCAL DEPLOYMENT · STEPS 1–5

Build once, then start five service containers

01 **Create .env**
Fill every secret with a fresh value

```
cp .env.example .env
```

02 **Build the reactor**
shared libs, 20 services, 4 apps; JDK 25

```
cd ji-hajoor   # repository root
mvn -q -DskipTests -Djacoco.skip=true package
```

03 **Rebuild after a change**
Only the deployable that hosts it

```
mvn -q -DskipTests -Djacoco.skip=true package \
  -pl apps/<deployable> -am
```

04 **Start Kafka and Redis**
Postgres runs as a Windows service

```
C:\kafka\bin\windows\kafka-server-start.bat C:\kafka\config\
server.properties
```

05 **Start the stack**
Waits on every health check

```
docker compose -f docker-compose.core.yml up -d --build --wait
```

Never keep the .env.example secrets: until 18 Oct it carried the JWT secret leaked in commit 1b5656d.

LOCAL DEPLOYMENT · STEPS 6–9

Seed data, prove the flows, open the apps

06 **Seed in this order**

Catalog, pricing, providers, agency

```
. docker/local-infra.sh
"$PSQL" -f docker/seed-catalog.sql
bash docker/seed-pricing.sh
"$PSQL" -f docker/seed-provider-profiles.sql
"$PSQL" -f docker/seed-tenants.sql
```

07 **Run the checks**

Booking flow, outbox path, sign-up

```
bash docker/smoke-flows.sh
bash docker/verify-outbox-flow.sh
bash docker/verify-signup.sh
```

08 **Open the apps**

Test users when DEV_SEED_ENABLED=true

```
:5173 customer · :5174 provider · :5175 admin
```

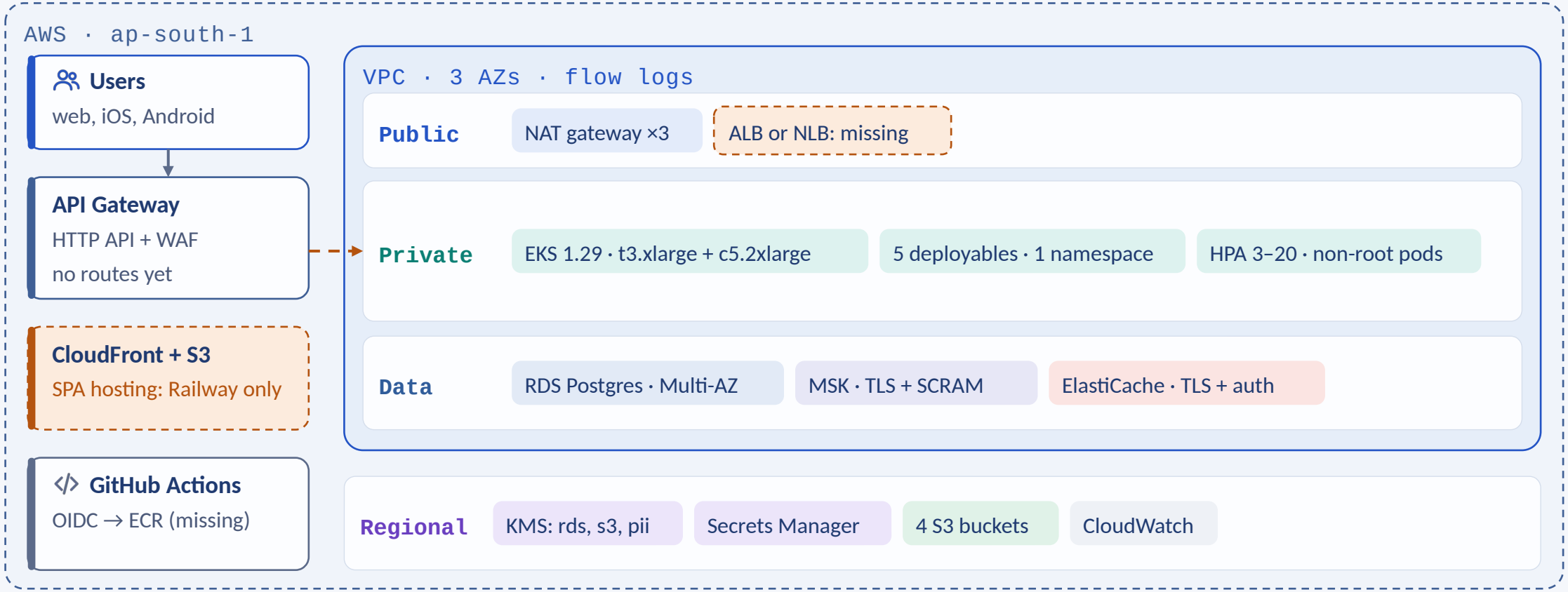
09 **Iterate on one app**

Vite proxies /api to the stack

```
cd frontend/customer-app
npm install && npm run dev
```

PRODUCTION DEPLOYMENT · TOPOLOGY

The data tier is ready; the path into the cluster is not



PRODUCTION DEPLOYMENT · STEPS 1–6

Provision AWS in two Terraform phases

01	**Clear the blockers** Start with the stray edit	`delete the stray '32wqa' on line 1 of api_gateway/main.tf`
02	**Bootstrap once** State, locks, ECR, OIDC roles	`jihajoor-terraform-state (versioned, KMS)` `jihajoor-terraform-locks (DynamoDB)`
03	**Phase 1: platform** Network, EKS, data, edge	`terraform init -backend-config=…` `terraform apply -target=module.kms -target=…`
04	**Phase 2: IAM, namespace** EKS endpoint and CA into tfvars	`terraform apply -target=module.iam -target=module.k8s_namespaces`
05	**Load real secrets** Replace the placeholders	`Secrets Manager → External Secrets → <svc>-secrets`
06	**Prepare the database** Per-service roles and schemas	`init-db.sql for RDS, then a Flyway Job`

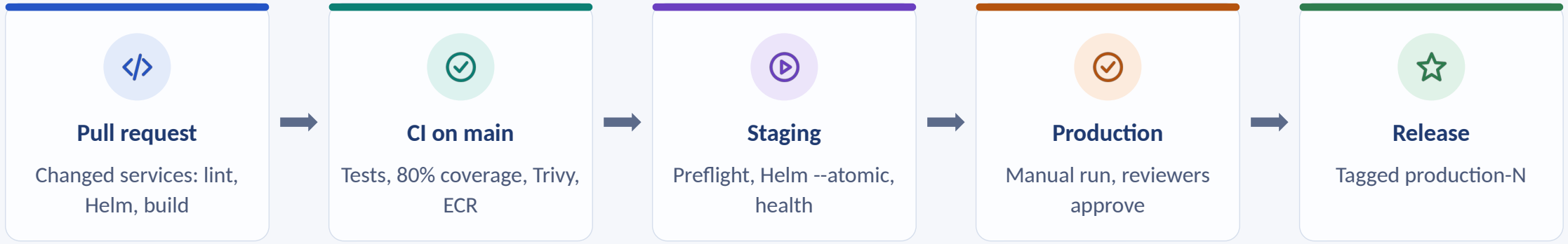
PRODUCTION DEPLOYMENT · STEPS 7–12

Ship a commit: staging first, production by approval

07	**Install cluster add-ons** LB controller, secrets, monitoring	`helm install … infra/k8s/observability/*-values.yaml`
08	**Merge to main** Tests, coverage, Trivy, ECR push	`vars: IMAGE_PUSH_ENABLED, STAGING_DEPLOY_ENABLED`
09	**Staging deploys itself** Preflight, then Helm per deployable	`helm upgrade --install jihajoor-<deployable> --atomic --wait`
10	**Promote to production** Same SHA; reviewers approve	`gh workflow run cd-production.yml -f image_tag=<sha> -f services=all`
11	**Publish the web apps** Railway builds each app	`frontend/<app>: nginx image from main (railway.json)`
12	**Watch, then roll back** Alarms, .DLT topics, errors	`re-run cd-production with the previous SHA`

DELIVERY PIPELINE

GitHub Actions carries a commit from PR to release



NOT AUTOMATED YET

- Terraform plan and apply
- Web app deploy
- Migrations as a Job
- IaC, secret, npm scans

PRODUCTION DEPLOYMENT

Ten blockers stand between Terraform and live traffic

#	Blocker	Fix
1	Stray "32wqa" in api_gateway/main.tf breaks plan	Delete it; it is committed
2	HTTP API has no routes, integrations or VPC Link	ALB ingress, or VPC Link + NLB
3	WAFv2 cannot attach to an HTTP API stage	Move WAF to the ALB or CloudFront
4	JWT authorizer needs JWKS; tokens are HMAC	Drop it, or move auth to RS256
5	No ECR, OIDC roles, state bucket or lock table	Add a bootstrap Terraform stack
6	Secrets are placeholders, never synced to K8s	External Secrets; align key names
7	Helm passes no Kafka SASL or Redis TLS settings	Wire SCRAM, TLS and auth tokens
8	Flyway runs in every replica at startup	Run migrations as a pre-deploy Job
9	Web apps are hosted only on Railway	S3 + CloudFront, or keep Railway
10	NetworkPolicies are not enforced by vpc-cni	Set enableNetworkPolicy

SECURITY · FIX TODAY

The secret that signed the leaked token must be rotated everywhere

Commit 1b5656d of the old home-fix repository is public on GitHub with that JWT_SECRET in it. .env.example stopped carrying it on 18 October, but any .env or environment still set to it accepts a SUPER_ADMIN token anyone can mint.

SECURITY ARCHITECTURE · TRUST BOUNDARIES

Each boundary has controls and at least one gap

Clients

Controls

- 15-min access tokens
- Refresh rotation

Gaps

- Refresh in localStorage
- No CSP or HSTS

Edge

Controls

- Token introspection
- Per-user rate limits

Gaps

- No per-IP login limit
- WAF can't attach

Services

Controls

- RBAC and ownership
- Keyed /internal APIs

Gaps

- One shared HMAC key
- Shared internal key

Data

Controls

- KMS at rest
- TLS Redis and MSK

Gaps

- RDS TLS not forced
- Pods read all secrets

Operations: EKS API open to 0.0.0.0/0; rotate the leaked JWT secret wherever it is still set.

SECURITY · STRENGTHS

The foundations are solid

Sessions

bcrypt 12, timing-safe login, rotating refresh tokens with replay detection

Access

Strict firewall closes path tricks; ownership checks answer 404

Payments

Razorpay HMAC plus read-back, signed webhook, server-side amounts

Internal APIs

Constant-time key check that fails closed; never routed at the edge

Data at rest

Private KMS-encrypted RDS, TLS Redis and MSK, locked down S3

Delivery

Non-root read-only pods, Trivy gate, OIDC to AWS, no secret defaults

Fixed since the last review: payment callback binding, location ownership, self-registration as ADMIN.

SECURITY FINDINGS · IDENTITY AND ACCESS

One Critical and four High findings sit in identity

Severity	Finding	Fix
Critical	The leaked JWT secret may still be set somewhere	Rotate per env; refuse dev values
High	One HMAC signing key is shared by all services	RS256 + JWKS; verify iss and aud
High	Services skip revocation; ports reachable directly	Gateway-only network; deny-list
High	No per-IP limit on password login or OTP requests	WAF rate rule on /auth; CAPTCHA
High	Gateway pairs Cloud 2025.1 with Boot 4.1	Pin Boot 4.0 or move to Cloud 2026.0
Medium	One INTERNAL_API_KEY opens every /internal API	Per-caller keys or mTLS

Together, the top three mean a leak from any one service, pod or log gives platform-wide SUPER_ADMIN.

SECURITY FINDINGS · MONEY AND OWNERSHIP

Razorpay is sound; the logic around it leaks money

Severity	Finding	Fix
High	Coupon caps never enforced; customers reset counts	Redeem server-side per booking
High	Support agents can approve any refund amount	Finance approval above a limit
Medium	Payment simulator selectable when enabled	Never enable it outside local
Medium	Payouts simulated, any amount, no idempotency	RazorpayX; amount from the wallet
Medium	Refunds leave the provider's earning in place	Debit the wallet on refund
Medium	Complaints accept any bookingId and providerId	Check the booking's participants
Medium	Booking accepts another customer's addressId	Verify the address owner
Medium	Agency admin can enlist any provider by mobile	Provider accepts an invitation

SECURITY FINDINGS · INFRASTRUCTURE

Infrastructure is too open in three places

Severity	Finding	Fix
High	EKS API endpoint is open to 0.0.0.0/0	Private, or VPN and CI ranges
High	Every pod can read every secret, even the RDS master	One secret policy per service
High	Local stack on 0.0.0.0; Redis has no password	Bind 127.0.0.1; set a password
Medium	RDS does not force TLS connections	rds.force_ssl = 1
Medium	Nodes lack IMDSv2 and explicit EBS encryption	Launch template with both
Medium	Terraform state will hold DB passwords in clear	KMS state bucket; ignore *.tfstate
Medium	Grafana default password and embedding allowed	existingSecret; embedding off
Low	Tag-pinned actions and images; EKS 1.29	Pin by SHA or digest; upgrade

SECURITY FINDINGS · WEB, APPS AND HYGIENE

The browser side needs defence in depth

Severity	Finding	Fix
Medium	30-day refresh token kept in localStorage	HttpOnly SameSite cookie
Medium	No CSP or HSTS; nginx containers run as root	Add headers; nginx-unprivileged
Medium	/metrics and /prometheus are public	Separate management port
Medium	Uploads trust client type; photos inline	Magic-byte allow-list
Medium	OTP codes hashed with SHA-256, salted by phone	HMAC with a server key
Low	Client IP parsed by hand from X-Forwarded-For	Trusted-proxy configuration
Low	Any role can create bookings; unmatched paths open	CUSTOMER rule; fail closed
Low	Tracked bin/, crash logs and scratch files	git rm; extend .gitignore
Low	Android allowBackup on, minify off	Disable backup; enable R8

SECURITY HARDENING · STEP BY STEP

Eight steps to a launch-ready security posture

01 · today

Rotate

New JWT, internal, KMS and webhook secrets; purge 1b5656d

02

Identity

RS256 with JWKS, iss and aud, refresh cookie, per-IP limits

03

Edge

WAF on the ALB, CSP and HSTS, CORS to app origins

04

Network

Private EKS endpoint, enforced policies, RDS TLS

05

Privilege

Per-service secret policies and keys; IMDSv2

06

Money

Coupons, refund debit, payouts, ID ownership

07

Pipeline

Gateway Boot/Cloud match, SHA pins, IaC and npm scans

08 · gate

Verify

Pen-test staging before the first real payment

NEXT STEPS

Fix the secret this week, the edge before launch

This week

- Delete the api_gateway stray text
- Rotate secrets; purge 1b5656d
- Bind local ports to 127.0.0.1
- Align gateway Boot and Cloud

Before launch

- Ingress, ECR, OIDC, state
- Secret sync, Kafka/Redis TLS
- RS256, per-IP auth limits
- Coupon, refund-debit, payout fixes
- Migration Job, web hosting

After launch

- Push tokens, KYC vendor
- Provider online/offline toggle
- Terraform in CI with scans
- Observability applied by CI